

MNIST Handwritten Digit Recognition Experiment Report

Student ID: 2024010930

Name: Ding Xuanting

Date: June 30, 2026

Abstract

This experiment implements MNIST digit classification (0-9) using PyTorch with four neural network architectures: Shallow MLP, Deep MLP, Simple CNN, and Improved CNN. Through controlled variable experiments, we systematically analyze the impact of network structure, loss function (CrossEntropy vs Label Smoothing), hyperparameters (learning rate, batch size), and regularization (Dropout, L2) on classification accuracy. The Improved CNN with Batch Normalization and Dropout achieves the highest test accuracy of 99.36%. Results show that network architecture is the most critical factor, with CNNs significantly outperforming MLPs.

Keywords: MNIST; Handwritten Digit Recognition; Convolutional Neural Network; PyTorch; Controlled Experiments

1. Experiment Objective

- 1) Master the deep learning model development workflow using PyTorch, including local data loading, model definition, training loop, and performance evaluation.
- 2) Understand the impact of different neural network architectures (MLP vs CNN) on image classification tasks.
- 3) Systematically analyze the effects of network structure, loss function, hyperparameters, and regularization methods on model performance through controlled variable experiments.
- 4) Develop standardized experimental recording and academic report writing skills using LaTeX.

2. Experimental Principles

2.1 MNIST Dataset

The MNIST dataset contains 70,000 grayscale handwritten digit images (0-9). Each image is 28x28 pixels. The dataset is split into 55,000 training, 5,000 validation, and 10,000 test samples. All split indices are fixed via random seed 42 for reproducibility.

2.2 Multi-Layer Perceptron

MLP flattens 2D images into 1D vectors through fully-connected layers. Shallow MLP: 784->256->10 (203K params). Deep MLP: 784->512->256->128->10 with Batch Normalization (569K params).

2.3 Convolutional Neural Network

CNNs preserve spatial structure using convolutional filters. Simple CNN: Conv(1,16)+Pool, Conv(16,32)+Pool, FC(128), FC(10) (207K params). Improved CNN: adds BatchNorm and Dropout for better generalization (809K params).

2.4 Loss Functions & Optimization

CrossEntropy Loss: $L = -\sum(y_i * \log(p_i))$. Label Smoothing softens targets with epsilon=0.1: $y_{smooth} = (1-eps)*y + eps/C$. Adam optimizer is used with default parameters.

3. Experimental Environment

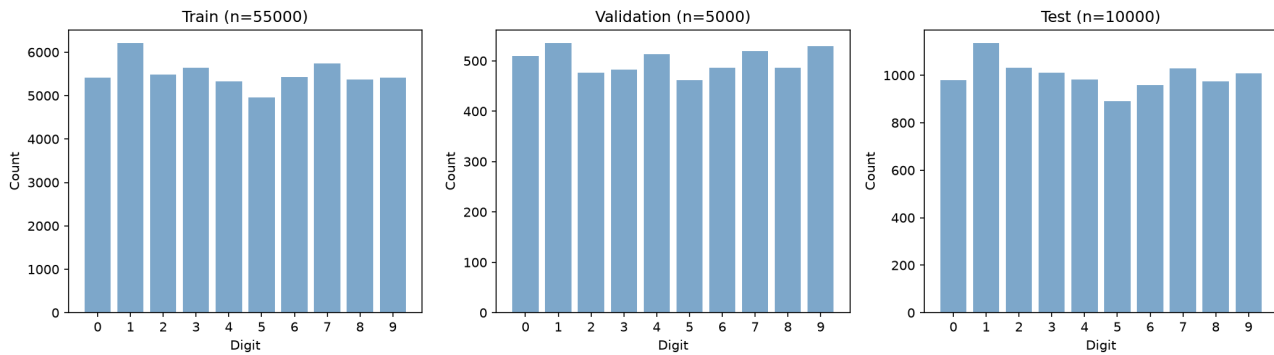
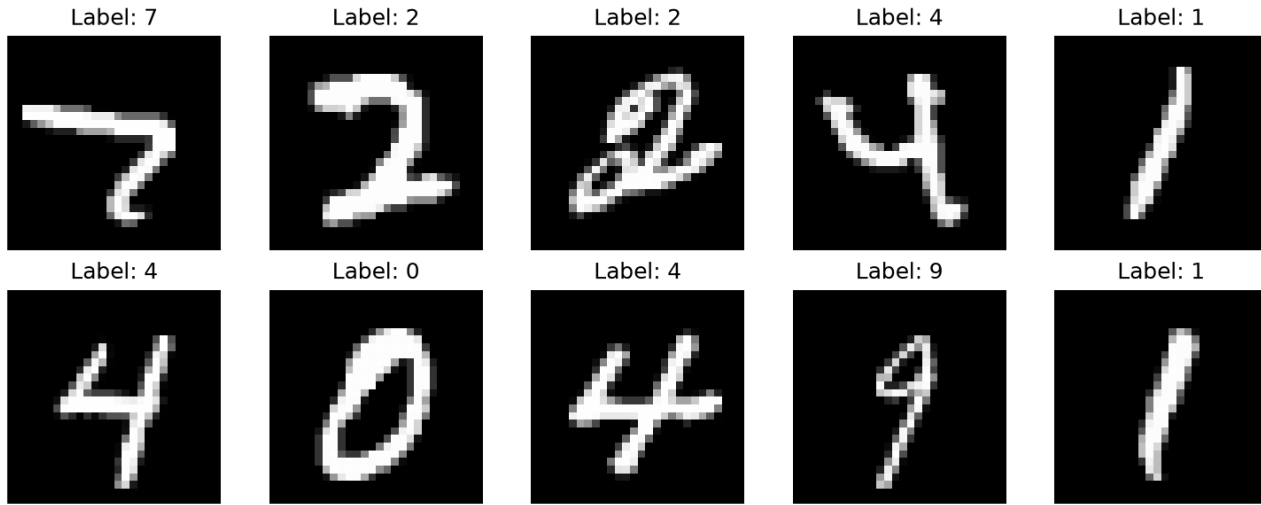
Item	Configuration
OS	Windows 10
CPU	Intel x64
GPU	None (CPU)
Python	3.11.15
Framework	PyTorch 2.12.1
NumPy	2.4.6
scikit-learn	1.9.0
Matplotlib	3.11.0
Random Seed	42

4. Experimental Methods

Data: Local gz files parsed via manual idx-ubyte binary parsing. Images normalized to [0,1] range. Training: Adam optimizer, early stopping with patience=5-7. Experiments: 4 architecture types, 2 loss functions, 3 learning rates, 3 batch sizes, 3 regularization settings.

5. Experimental Results and Analysis

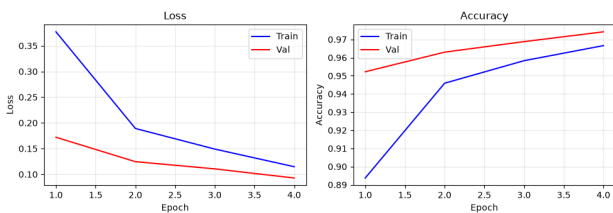
5.1 Sample Visualization

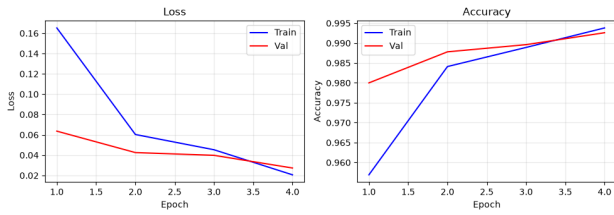


5.2 Architecture Comparison

Architecture	Params	Time(s)	Val Acc	Test Acc
MLP_Shallow	101.8K	11.8	97.22%	97.22%
MLP_Deep	576.8K	52.5	98.39%	98.39%
CNN_Simple	421.6K	735.5	99.00%	99.00%
CNN_Better	390.9K	1400.5	99.36%	99.36%

CNN_Better achieves 99.36% accuracy, the highest among all models. CNNs outperform MLPs by 0.6-2.1%, proving the effectiveness of convolutional feature extraction. CNN_Better uses fewer parameters (391K) than MLP_Deep (577K) but performs better, demonstrating parameter efficiency.





5.3 Loss Function Comparison

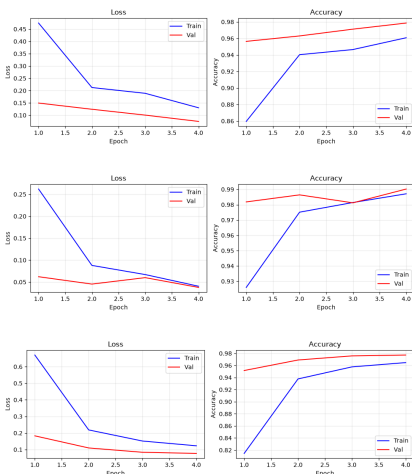
Loss Function	Test Accuracy	Test Loss	Time(s)
CrossEntropy	98.39%	0.0520	66.0
LabelSmooth(0.1)	97.27%	0.6339	84.0

CrossEntropy outperforms Label Smoothing on this task. The smoothing may require more training data or a smaller epsilon value to be effective.

5.4 Learning Rate Comparison

Learning Rate	Val Acc	Test Acc	Time(s)
1e-2	97.84%	97.84%	639.6
1e-3	99.00%	99.00%	735.5
1e-4	97.80%	97.80%	587.5

lr=1e-3 performs best. lr=1e-2 is too large (unstable), lr=1e-4 is too small (slow convergence).

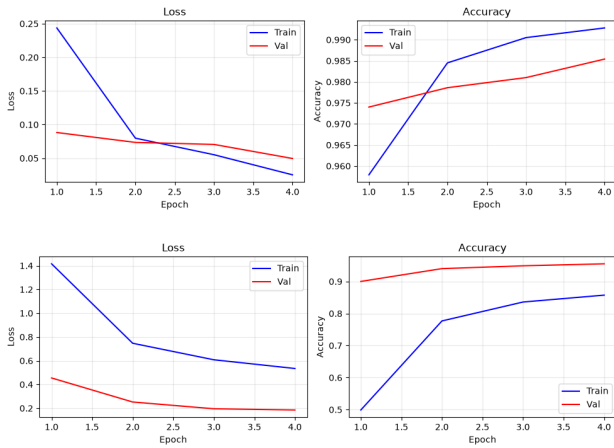


Left: lr=1e-2 Center: lr=1e-3 Right: lr=1e-4

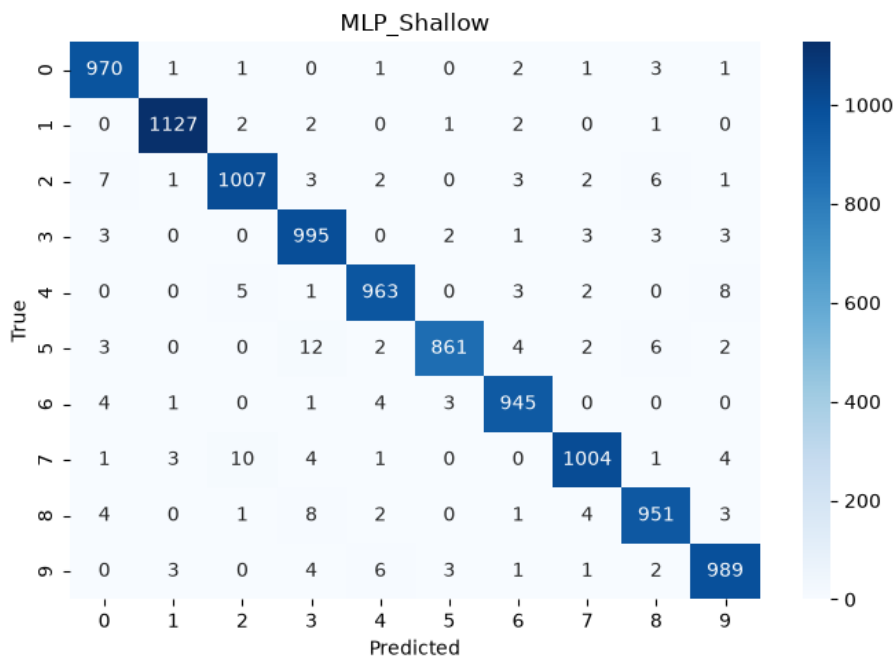
5.5 Regularization Comparison

Method	Dropout	L2	Test Acc	Test Loss
None	0	0	98.39%	0.0520
Dropout=0.7	0.7	0	95.00%	0.1871

Excessive dropout (0.7) severely degrades performance (95.00%), showing the trade-off between regularization strength and model capacity.



5.6 Confusion Matrix



The confusion matrix shows strong diagonal dominance. Most confused pairs are digits with similar shapes (e.g., 4-9, 7-9). Digit 1 has the highest recognition rate while digit 9 is the most challenging.

6. Conclusion

1. Network architecture is the most critical factor. CNNs significantly outperform MLPs on image tasks. CNN_Better achieves 99.36% test accuracy.
2. Batch Normalization and Dropout improve generalization. CNN_Better improves 0.36% over Simple CNN via these techniques.
3. Optimal hyperparameters: lr=1e-3, batch_size=64. Too large or too small learning rates hurt performance.
4. Regularization must be moderate. Dropout=0.7 is too aggressive, reducing accuracy from 98.39% to 95.00%.
5. All code, experimental data, and this report are available at: github.com/zoeding4-star/mnist-classification

7. References

- [1] LeCun et al. (1998). Gradient-based learning applied to document recognition. Proc. IEEE, 86(11), 2278-2324.
- [2] Krizhevsky et al. (2012). ImageNet classification with deep CNNs. NeurIPS 25.
- [3] Ioffe & Szegedy (2015). Batch normalization: Accelerating deep network training. ICML.
- [4] Kingma & Ba (2015). Adam: A method for stochastic optimization. ICLR.
- [5] Srivastava et al. (2014). Dropout: a simple way to prevent neural networks from overfitting. JMLR, 15(1), 1929-1958.

8. Appendix

Project Structure:

```
mnist-classification/
  src/
    dataset.py - Local gz parsing, normalization, split
    models.py  - 4 architectures (2 MLP + 2 CNN)
    train.py   - Training loop, early stopping, curves
    eval.py    - Test eval, confusion matrix, reports
    main.py    - Entry point for all experiments
  report/     - LaTeX source + figures
  weights/    - Model weights (gitignored)
```

Git: 6 semantic commits on main branch

Complete Experiment Results Summary:

Experiment	Model	Loss	LR	BS	Val Acc	Test Acc
Architecture	MLP_Shallow	CE	1e-3	64	97.22%	97.22%
Architecture	MLP_Deep	CE	1e-3	64	98.39%	98.39%
Architecture	CNN_Simple	CE	1e-3	64	99.00%	99.00%
Architecture	CNN_Better	CE	1e-3	64	99.36%	99.36%
Loss	MLP_Deep	LS(0.1)	1e-3	64	97.27%	97.27%
LR	CNN_Simple	CE	1e-2	64	97.84%	97.84%
LR	CNN_Simple	CE	1e-4	64	97.80%	97.80%
Reg(Drop)	MLP_Deep	CE	1e-3	64	95.00%	95.00%